Below is a **drop-in first doc** you can place into your repo as `docs/data_integrity_and_leakage.md` (or `docs/leakage.md`). It is written to be "reviewer-grade": explicit about what is guaranteed, what was previously wrong, what changed, and what remains unknown.

Data Integrity and Leakage Controls

This document explains how the current codebase prevents **information leakage** in both training and evaluation for the **AlphaModel** (run100/run101/run102 lineage; deployed as run123 checkpoint) and the downstream **PolicyModel** (run133 lineage). It also records the specific leakage vectors discovered earlier and how they were fixed.

The goal is to make it easy for reviewers and users to verify that:

- **Train/val/test are time-disjoint**
- **Labels are aligned (t → t+1)**
- **No val/test features are visible during training**
- **Normalization / scaling is fit on train only**
- **Attention mechanisms cannot see the future**
- **OOS evaluation sets are leakage-safe**
- **Live inference path does not reintroduce leakage**

If something is not explicitly documented in the repo, it is listed in **Unknowns / Not Yet Documented**.

---

1. Definition: what counts as leakage here?

In this codebase, "leakage" includes any of the following:

1. **Lookahead in labels** Using information from day *t+1* (or later) to build day t features or to train day t predictions.
2. **Split contamination in preprocessing** Computing normalization statistics (mean/std) on train+val+test, then applying them during training (val/test information influences train).
3. **Feature exposure during training** Even if the **loss** is masked to train-only, leakage still occurs if the **model inputs** include val/test features (because parameters can adapt to future regimes and patterns).
4. **Non-causal modeling paths** Attention masks or data alignment that allow a token at time t to attend to or incorporate information from time *t+1*.

---

2. Leakage issues found historically

Two concrete leakage vectors were identified in the Run101 cross-ticker pipeline:

2.1 Normalization leak

Earlier dataset builds computed `price_mean/std` and `news_intensity_mean/std` over **all dates** (train+val+test). These stats were stored in the `.pt` records and used during training, allowing future information to influence scaling.

2.2 Feature exposure leak in cross-ticker dataset

`Run101MultiTickerDataset` originally **masked targets by split**, but still fed **val/test features** into the `split=train` training loader. This is leakage because the model can learn from future-period covariates even if the loss ignores future labels.

Repo reference: `docs/run101_leak_fix.md`

---

3. Fixes applied (current codebase)

3.1 Train-only normalization stats

The build pipeline was updated so that price and news intensity normalization are computed using **train dates only**.

- Train-only cutoff is the end of train split: $\leq$ **2024-06-30**
- This applies to:
  - `price_mean/std`
  - `news_intensity_mean/std`
  - (and factors/targets follow the same rule per the runbooks)

Repo references:

- `docs/run101_leak_fix.md`
- `docs/run102_runbook.md`

3.2 Split-aware feature masking in Run101MultiTickerDataset

The cross-ticker dataset now applies the split mask to **both**:

- Targets (labels)
- Features (price/news/ticker masks and any stacked tensors)

Concretely, for `split=train`, val/test portions of:

- `price_seq_stack`
- `price_hidden_stack`
- `news_emb_stack`

- `news_mask_stack`
- `news_intensity_stack`
- `ticker_mask_stack`

…are masked out so the model cannot "see" future features.

Repo references:

- `docs/run101_leak_fix.md`
- `docs/run102_network_overview.md`

---

4. Splits: time-based and disjoint (train/val/test)

Splits are **purely time-based** and date-bounded:

- **Train:** dates $\leq$ **2024-06-30**
- **Val: 2024-07-01** to **2024-09-30**
- **Test: 2024-10-01** to **2024-12-31**

Split masks are checked to be disjoint in the label sanity audit, and per-epoch evaluation is typically restricted to train/val to avoid test peeking during development runs.

Repo references:

- `docs/run101_runbook.md`
- `docs/run101_label_debug.md`
- `docs/run102_runbook.md`

---

5. Label alignment (lookahead prevention)

5.1 Alpha labels are next-day

The forecasting task is aligned as:

- Inputs at date **t** $\rightarrow$ predict return at **t+1**
- The last day is masked because **t+1** is not available

This is implemented in the run102/run100 lineage as "next-day labels" and used consistently in training and evaluation.

Repo references:

- `docs/run101_leak_fix.md`
- `docs/run102_runbook.md`
- `docs/run102_network_overview.md`

5.2 Policy execution timing is also next-day

The policy model selects weights at **t**, and those weights apply to returns at **t+1** (consistent with how alpha `mu[t]` is intended to be consumed).

This is explicitly documented in the run133/run134 pipeline.

Repo references:

- Run133 policy overview (in your internal notes)
- `docs/run134_inference_commands.md`

---

6. Feature masking rules (what the model is allowed to see)

For training on a given split, the system is designed so that **both labels and inputs are gated**.

6.1 Full-seq single-ticker mode (Run100PTDataset / Fullseq)

- Split masks (`split_mask`) exist per day.
- Training loss is computed only on valid tokens for the active split.
- Padded positions are also masked.

6.2 Cross-ticker mode (Run101MultiTickerDataset)

- Split masks gate:
  - targets *and*
  - all feature tensors
- Ticker padding masks hide inactive / missing tickers.

Repo references:

- `docs/run102_network_overview.md`
- `docs/run101_leak_fix.md`

---

7. Model causality (no-future constraints inside the network)

The architecture includes multiple attention components. The current design applies "no future" constraints as follows:

7.1 TimeLLM (per ticker)

- **Causal along time**
- Cannot attend to future days within a ticker

7.2 Factor cross-attention

- Uses a **causal mask** so query day **t** cannot attend to factor tokens from **t+1+**
- Typically described as "k < q" (or "day_idx ≤ t") constraint

7.3 Cross-ticker attention

- Operates **within the same day** across tickers
- Does not introduce temporal lookahead because it is cross-sectional per time step
- Uses `ticker_mask` to hide padded / inactive tickers

Repo references:

- `docs/run101_runbook.md`
- `docs/run102_network_overview.md`

---

8. Leakage-safe out-of-sample (OOS) evaluation sets

8.1 "Golden" OOS set (run102 golden)

A separate golden evaluation set is maintained as **test-only**, built using:

- train-only normalization stats
- the same bin edges / normalization conventions as the corrected pipeline

This exists to provide a leakage-resistant evaluation path distinct from the train/val/test windows used during model development.

Repo reference: `docs/run102_network_overview.md`

8.2 Pre-2019 OOS window

The project also uses a pre-2019 OOS window (2010–2018) as an additional "regime" test. This is **chronologically non-overlapping** with the 2019–2024 training regime used in the later pipeline.

(Exact build details depend on your golden manifest builder and bin-edge pinning choices; see "Unknowns" below for what should be made explicit in public docs.)

---

9. Live inference path and news leakage

9.1 Live inference uses price-only sequences

In run134 live inference, the pipeline uses:

- **price-only run101 sequences**
- no news embeddings

This materially reduces the surface area for news timestamp leakage in the live deployment path.

Repo reference: `docs/run134_inference_commands.md`

---

10. Guardrails to prevent leakage regressions

These are the rules contributors should follow to avoid reintroducing leakage:

1. **Never fit normalization on val/test** `price_mean/std` and `intensity_mean/std` must be computed on train dates only.
2. **Split masks must gate features, not just labels** Particularly in cross-ticker or multi-asset stacks, feature exposure is a real leakage vector.
3. **Keep label timing explicit and consistent**
   - Alpha predicts t→t+1
   - Policy uses weights chosen at t applied at t+1
4. **Do not "peek" at test during iteration**
   - Prefer per-epoch eval on train/val only
   - Run test/golden only for milestone checkpoints
5. **Pin bin edges and universe**
   - Recomputing bins or changing ticker universe can create silent distribution shifts that look like "performance changes."
   - Keep bins stable across rebuilds (or make rebuild explicit and versioned).

---

11. Known unknowns / not yet documented (public-facing gaps)

These items are not fully specified in the current repo docs (or not explicitly spelled out) and should be documented to strengthen credibility:

1. **News timestamp cutoff / embargo rules**
   - If/when news is used: what qualifies as "available by end of day t"?
   - Are post-close articles included for day t?
2. **Automated leakage tests**
   - There is no explicitly documented CI/unit-test suite that asserts:

  - disjoint split masks,
  - train-only normalization,
  - feature masking correctness in cross-ticker dataset.

3. **Formal leakage audit for run123/run133**
   - The fixes are applied in the pipeline lineage, but a "run123-specific audit checklist" is not yet written.
4. **Kronos precompute causality guarantees**
   - The Kronos path is frozen and used as an encoder, but the public docs should clarify that inputs are strictly day t features only and do not contain forward-filled labels.

---

12. Recommended next steps (to make this reviewer-proof)

If you want this project to withstand skeptical review (quant funds + applied science interview loops), add a lightweight "Leakage Test Suite":

12.1 Add a script: `scripts/audit_no_leakage.py`

It should assert:

- Split masks are disjoint: `(train & val).any() == False`, etc.
- Normalization tensors were computed only over train timestamps (or at least that the builder used the train-only mode).
- Cross-ticker dataset masks **features** outside the split (spot-check several tensors).
- Label alignment sanity:
  - last day masked,
  - no nonzero labels where future isn't available.

12.2 Add a CI job (even if optional)

- `python -m pytest -k leakage`
- Or a single "audit" run that executes quickly on CPU.

This doesn't need to be heavy—just enough to prevent accidental regressions.

Appendix A: Leakage-focused Q&A (summary)

Q: What leakage issues were found? A: Two: (1) normalization computed on train+val+test, and (2) cross-ticker dataset fed val/test features into train loader even though labels were masked.

Q: How were they fixed? A: Train-only normalization stats and split-aware masking of **both** targets and features in `Run101MultiTickerDataset`.

Q: Are splits time-based and disjoint? A: Yes. Train $\leq$ 2024-06-30, val 2024-07-01..09-30, test 2024-10-01..12-31.

Q: Are labels aligned to prevent lookahead? A: Yes. Next-day labels (t$\rightarrow$t+1); last day masked.

Q: Are model components causal? A: Yes: TimeLLM is causal over time; factor cross-attn uses causal masking; cross-ticker attention is cross-sectional at the same day only; padding masks hide inactive tickers.

Q: Is news a leakage risk in current live inference? A: Not in run134 live, which uses price-only sequences and no news embeddings.

References (repo docs)

- `docs/run101_leak_fix.md`
- `docs/run101_runbook.md`
- `docs/run101_label_debug.md`
- `docs/run102_network_overview.md`
- `docs/run102_runbook.md`
- `docs/run134_inference_commands.md`

If you want, I can also write the companion `scripts/audit_no_leakage.py` spec (not code yet, just a crisp checklist + expected asserts), so the repo has an executable "trust gate" rather than only narrative documentation.